

|                   |                 |                    |                   |                 |                        |
|-------------------|-----------------|--------------------|-------------------|-----------------|------------------------|
| 10<br>CONTROLLERS | 46<br>ENDPOINTS | 12<br>JPA ENTITIES | 20<br>REACT PAGES | 6<br>USER ROLES | 13.5k<br>LINES OF CODE |
|-------------------|-----------------|--------------------|-------------------|-----------------|------------------------|

PROJECT OVERVIEW

WorkGate is an internal employee portal modelled on FDM Group’s consultant operating model. It replaces the scattered spreadsheets and email chains of day-to-day HR admin with a single authenticated workspace where **the interface reshapes itself around who is logged in**.

An employee books annual leave, files an expense claim, raises an IT ticket and tracks assigned tasks. Their manager sees those same records as an **approval queue** with accept/reject actions and running leave balances. HR handles confidential reports including anonymous submissions, IT triages tickets and unlocks accounts, and an administrator provisions staff, sets reporting lines and manages client billing codes. Six roles, one codebase, one navigation shell.

ARCHITECTURE

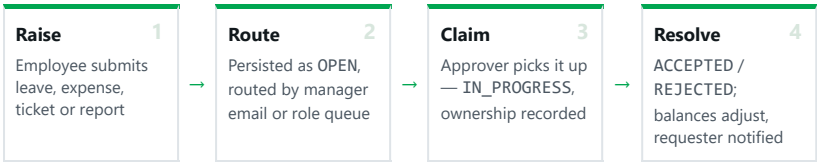
|                                 |                                    |
|---------------------------------|------------------------------------|
| <b>Backend</b> · Spring Boot    | <b>Frontend</b> · React + Vite     |
| controller/ 10 REST controllers | pages/ 20 route components         |
| model/ JPA entities + enums     | components/ Layout, Sidebar, Modal |
| repository/ Spring Data JPA     | context/ Auth + Theme providers    |
| DTO/ 14 request/response types  | api/ single fetch wrapper          |
| config/ CORS policy             | permissions.js RBAC map            |

The domain model uses **single-table inheritance** in both directions: User → Employee → Consultant on the people side, and Request → ManagerRequest → AnnualLeaveRequest / ExpenseRequest on the workflow side, using Hibernate discriminator columns. Shared approval logic lives once on the abstract parent rather than per request type. The React permission map deliberately **mirrors that same Java hierarchy**, so a role’s UI capabilities and its domain capabilities stay in step.

ROLE CAPABILITIES

| ROLE       | UNLOCKS                                                                                   |
|------------|-------------------------------------------------------------------------------------------|
| Employee   | Dashboard, profile, tasks, leave requests, expenses, IT & HR support, news feed           |
| Consultant | Employee base + client-code assignment and deployment status (trainee / bench / deployed) |
| Manager    | Employee base + leave approval, expense approval, task assignment, news posting           |
| HR         | Employee base + employee-report management, claim & resolve workflow, news posting        |
| IT Tech    | Employee base + ticket triage, priority handling, account unlock after lockout            |
| Admin      | Separate branch — provision & deactivate staff, reporting lines, client codes             |

REQUEST LIFECYCLE



Every request type shares this state machine via the abstract Request entity — a new workflow costs one subclass and one controller, not a new pipeline.

SELECTED API SURFACE

|      |                                     |                                     |
|------|-------------------------------------|-------------------------------------|
| POST | /api/login                          | Authenticate, return role tag       |
| POST | /api/createAnnualLeave              | Raise leave request against balance |
| GET  | /api/annualLeave/manager            | Approval queue for a manager        |
| POST | /api/resolveExpenseRequest          | Approve or reject with reason       |
| PUT  | /api/admin/employees/{email}/unlock | Clear lockout                       |

TECHNOLOGY

|          |                                                                             |
|----------|-----------------------------------------------------------------------------|
| LANGUAGE | Java 21 · JavaScript ES2022                                                 |
| BACKEND  | Spring Boot 3.5.12 — Web MVC, Data JPA, Hibernate 6.6                       |
| DATABASE | SQLite via xerial JDBC 3.46 + community dialect; schema auto-managed        |
| FRONTEND | React 18.3 · React Router 6.26 · Context API                                |
| BUILD    | Vite 8 dev server with HMR · Maven with wrapper                             |
| STYLING  | CSS Modules + custom properties — no UI framework, ~4.3k hand-written lines |
| TESTING  | Spring Boot Test / JUnit                                                    |

ENGINEERING HIGHLIGHTS

- **Zero UI dependencies.** Every component, modal, grid and dark-mode token is hand-built — three runtime packages in total.
- **Inheritance-driven design.** Polymorphic JPA hierarchies keep shared approval behaviour in a single place.
- **Declarative route guarding.** <RoleRoute permission="..."> wraps each route against a central permission map.
- **Account lockout.** Five failed logins lock a user and only IT can reset — modelled in the domain, not bolted on.
- **Full-stack CRUD depth.** Leave, expenses, tickets, reports, tasks, posts and client codes each carry a complete lifecycle.
- **Graceful degradation.** A mock-data fallback lets the UI be demoed with the API offline.

RUNNING IT

```
# backend → :8080
$ cd WorkGate-backend/FDM
$ ./mvnw spring-boot:run

# frontend → :5173
$ cd WorkGate-frontend/FDM
$ npm install && npm run dev
```

SQLite provisions itself on first boot — no database setup, no container, no seed script.

SCOPE & NEXT STEPS

**HONEST SCOPE**

Built as an academic full-stack exercise, so authentication is intentionally simple: credentials are compared directly and identity is held client-side. Production hardening is the clear next milestone — **BCrypt hashing, Spring Security with JWT sessions, server-side authorisation on every endpoint**, and a move from SQLite to PostgreSQL with Flyway migrations. The domain model and API surface are already shaped to absorb that layer without restructuring.